

LINQ Reverse Method in C# with Examples

In this article, I will discuss the **LINQ Reverse Method in C#** with Examples. Please read our previous article before proceeding to this article, where we discussed [LINQ ThenBy and ThenByDescending Method in C#](#) with Examples. As part of this article, we will discuss the following pointers in detail.

1. **What is the Reverse Method in C#?**
2. **Different Types of Examples using both Method and Query Syntax.**
3. **We will discuss the Reverse Method available in the System.Linq and System.Collections.Generic namespaces.**
4. **Finally, we will discuss applying the LINQ Reverse method (the Reverse Method available in System.Linq namespace) on a List<T> type collection.**
5. **When to Use the LINQ Reverse Method?**

What is the Reverse Method in C#?

The Reverse method in LINQ is used to invert the order of elements in a collection in C# and other .NET languages. It's a straightforward and useful method for reversing the sequence of the elements in an array, list, or any other collection that implements the `IEnumerable<T>` interface without needing manual looping or additional logic. This method will not change the data; rather, it simply reverses the data stored in the data source. As a result, we will get the output in reverse order.

Reverse Method in C#:

The Reverse Method in C# is implemented in two different namespaces. This method is implemented in **System.Linq** namespace as well as in **System.Collections.Generic** namespaces. Let us look at the signature or definition of the Reverse Method, shown in the image below.

```
namespace System.Linq
```

```
public static IEnumerable<TSource> Reverse<TSource>(this IEnumerable<TSource> source);
```

```
namespace System.Collections.Generic
```

```
public void Reverse();
```

As you can see, the Reverse Method belongs to **System.Linq** namespace is implemented as an extension method on the `IEnumerable<TSource>` interface, and more importantly, this method also returns an `IEnumerable<TSource>` type. On the other hand, the Reverse method belongs to the **System.Collections.Generic** namespace is not returning any value as the return type is void. With this kept in mind, let us see some examples for a better understanding of the Reverse method in C#.

Example to Understand System.Linq Reverse Method in C#

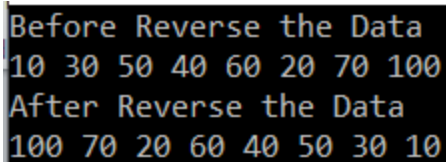
Let us see an example of Understanding the **System.Linq** namespace Reverse Method in C#. Please have a look at the below example. In the example, we created one integer array to store integer numbers. Then,

we call the Reverse method on the intArray, which will return a new collection of int type with data in reverse order. Then, the data was printed using a for each loop.

```
using System.Collections.Generic;
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] intArray = new int[] { 10, 30, 50, 40, 60, 20, 70, 100 };
            Console.WriteLine("Before Reverse the Data");
            foreach (var number in intArray)
            {
                Console.Write(number + " ");
            }
            Console.WriteLine();
            IEnumerable<int> ArrayReversedData = intArray.Reverse();
            Console.WriteLine("After Reverse the Data");
            foreach (var number in ArrayReversedData)
            {
                Console.Write(number + " ");
            }

            Console.ReadKey();
        }
    }
}
```

Output:



```
Before Reverse the Data
10 30 50 40 60 20 70 100
After Reverse the Data
100 70 20 60 40 50 30 10
```

Note: In the above example, if you go to the definition of the Reverse method, then you will see that this Reverse method belongs to the **System.Linq** namespace, we can store the data in a variable of type `IEnumerable<int>` as the source contains integer data.

Example to Understand System.Linq Reverse Method with Query Syntax:

Let us see an example to Understand the System.Linq Reverse Method with Query Syntax. The most important point you need to remember is that we do not have any such operator called Reverse available in Linq to write Query Syntax. So, here, we need to use Mixed syntax. Let us rewrite the previous example using Mixed Syntax.

```

using System.Collections.Generic;
using System;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            int[] intArray = new int[] { 10, 30, 50, 40,60,20,70,100 };
            Console.WriteLine("Before Reverse the Data");
            foreach (var number in intArray)
            {
                Console.Write(number + " ");
            }
            Console.WriteLine();
            IEnumerable<int> ArrayReversedData = (from num in intArray
                                                select num).Reverse();
            Console.WriteLine("After Reverse the Data");
            foreach (var number in ArrayReversedData)
            {
                Console.Write(number + " ");
            }

            Console.ReadKey();
        }
    }
}

```

Output:

```

Before Reverse the Data
10 30 50 40 60 20 70 100
After Reverse the Data
100 70 20 60 40 50 30 10

```

Example to Understand System.Collections.Generic Reverse Method in C#

Let us see an example to Understand the System.Collections.Generic namespace Reverse Method in C#. To understand the System.Collections.Generic Reverse method, we need to create a collection of List<T> types. Please have a look at the below example. Here, we created one list collection to store string values. Then, we call the reverse method on the stringList collection, which will reverse data in the original collection. Then, the original collection data was printed using a for each loop. The example below uses the Reverse method, which is defined inside the System.Collections.Generic namespace.

```

using System.Collections.Generic;
using System;
using System.Linq;
namespace LINQDemo

```

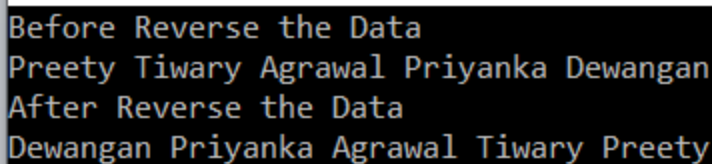
```

{
    class Program
    {
        static void Main(string[] args)
        {
            List<string> stringList = new List<string>() { "Preety", "Tiwary",
"Priyanka", "Dewangan"};

            Console.WriteLine("Before Reverse the Data");
            foreach (var name in stringList)
            {
                Console.Write(name + " ");
            }
            Console.WriteLine();
            //You cannot store the data like below as this method belongs to
            //System.Collections.Generic namespace whose return type is void
            //IEnumerable<int> ArrayReversedData = stringList.Reverse();
            stringList.Reverse();
            Console.WriteLine("After Reverse the Data");
            foreach (var name in stringList)
            {
                Console.Write(name + " ");
            }
            Console.ReadKey();
        }
    }
}

```

If you go to the definition of the above Reverse method, you can see this method belongs to the System.Collections.Generic namespace and the return type is Void. Now run the application, and you will get the output as expected, as shown in the below image.



```

Before Reverse the Data
Preety Tiwary Agrawal Priyanka Dewangan
After Reverse the Data
Dewangan Priyanka Agrawal Tiwary Preety

```

How to Apply the LINQ Reverse Method on a Collection of List<T> Type in C#?

Suppose you want to apply the Reverse method, which belongs to **System.Linq** namespace on a collection of type **List<T>**, then first you need to convert the **List<T>** collection to **IEnumerable** or **IQueryable** by using the **AsEnumerable()** or **AsQueryable()** method on the data source.

If you use the **AsEnumerable()** method, it will convert the collection to **IEnumerable**, whereas if you use the **AsQueryable()** method, it will convert the collection to **IQueryable**. The following program shows how to apply the LINQ Reverse method on a type List<T> collection.

```

using System.Collections.Generic;
using System;

```

```

using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<string> stringList = new List<string>() { "Preety", "Tiwary",
"Priyanka", "Dewangan" };
            Console.WriteLine("Before Reverse the Data");
            foreach (var name in stringList)
            {
                Console.Write(name + " ");
            }
            Console.WriteLine();
            IEnumerable<string> ReverseData1 = stringList.AsEnumerable().Reverse();
            IQueryable<string> ReverseData2 = stringList.AsQueryable().Reverse();
            Console.WriteLine("After Reverse the Data");
            foreach (var name in ReverseData1)
            {
                Console.Write(name + " ");
            }
            Console.ReadKey();
        }
    }
}

```

Output:

```

Before Reverse the Data
Preety Tiwary Agrawal Priyanka Dewangan
After Reverse the Data
Dewangan Priyanka Agrawal Tiwary Preety

```

Note: The LINQ Reverse Method in C# will return the data as `IEnumerable<TSource>` or `IQueryable<TSource>` based on how we use the LINQ method.

When should you use the LINQ Reverse Method in C#?

The LINQ Reverse method is used to invert the order of elements in a collection. It is a straightforward but powerful method, suitable for various scenarios:

- **Creating a Reverse Order View:** When you have a sequence of elements, and you need to process or display them in reverse order. For example, displaying a list of messages in reverse chronological order.
- **Reversing User-Defined Orderings:** If you've previously ordered a collection using `OrderBy` or similar methods and need to view or process the data in the exact opposite order, `Reverse` is a quick way to achieve this.

- **Stack-Like Behavior:** In scenarios where you need to process the last elements first (similar to a stack), Reverse can turn a queue-like collection (first-in, first-out) into a stack-like collection (last-in, first-out).
- **Simplifying Palindrome Checks:** In some string or array processing tasks, like checking for palindromes, reversing the sequence and comparing it to the original can be a straightforward approach.
- **UI and Visualization:** In user interfaces or data visualizations where the presentation order is crucial and differs from the stored order, Reverse can be used to easily flip the order of elements.
- **Dealing with Ordered Data from External Sources:** If data is received in an order that is inverse to what is needed (for instance, oldest-to-newest when newest-to-oldest is required), Reverse can quickly rectify this.

In the next article, I will discuss the [Linq Aggregate Methods in C#](#) with Examples. I hope this article gives you a good understanding of using the **Reverse Method** in C# with Examples.